

LLM 预训练之 Backpropagation

Gradient Flow / Autograd / Activation Recomputation / Distributed Backward

Deep Research Manual · 2026-08

核心判断 Backpropagation 的目标不是“把 Loss 往回传”这么简单，而是高效计算 $\nabla \theta L$ 。现代 LLM 依赖 reverse-mode automatic differentiation：沿计算图执行 vector-Jacobian product (VJP)，避免显式构造巨大 Jacobian。真正的工程问题随后变成：保存哪些激活、何时累积梯度、如何跨并行维度同步，以及如何保证数值稳定。

本册与相邻模块的边界

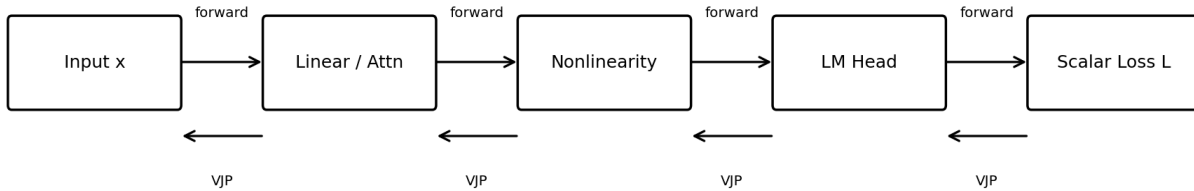
模块	回答的问题	本册是否展开
Loss / Objective	监督信号如何定义？哪些 token 计入 loss？	作为 backward 的起点，简述
Backpropagation	每个参数对 loss 的局部责任是多少？如何得到 $\nabla \theta L$ ？	本册核心
Optimizer	拿到梯度后，参数到底更新多少？	下一册
Distributed Runtime	梯度在哪些 GPU 上产生、聚合、分片？	只讨论 backward 相关部分

阅读路线

- 先理解 reverse-mode VJP 与计算图；
- 再理解 Transformer Block 内部的梯度流；
- 随后看 activation memory / checkpointing；
- 最后进入 mixed precision、gradient accumulation 与 DP/TP/PP/FSDP backward。

1. Backprop 的数学核心：Reverse-mode VJP

设模型输出 $y=f(x)$ ，最终 loss 为标量 L 。链式法则写成 Jacobian 形式： $dL/dx = J_{f(x)}^T \cdot dL/dy$ 。对 LLM 这种“参数非常多、最终只有一个 scalar loss”的系统，reverse-mode AD 特别合适：一次反向遍历即可得到所有参数梯度。



Reverse mode: $\bar{x} = J^T \bar{y}$; no full Jacobian is materialized

图 1 Reverse-mode AD: 反向传播的基本操作是 VJP，而不是显式 materialize 完整 Jacobian。

概念	数学对象	工程含义
Jacobian J	$\partial y / \partial x$	通常巨大，不显式存储
VJP	$v^T J$ 或 $J^T v$	backward 的核心计算原语
Leaf gradient	$\partial L / \partial \theta$	写入 parameter.grad / main_grad
Gradient accumulation	$g \leftarrow g + g_micro$	多个路径或 microbatch 的梯度相加

Know-why 为什么 backward 不会因为数十亿参数而构造一个“数十亿 × 数十亿”的矩阵？因为自动微分保存局部操作所需的信息，并逐节点执行 VJP；复杂度通常与一次或数次 forward 同阶，而不是 Jacobian 维度平方级。

1.1 一个 Linear 层的 backward

若 $Y = XW^T$ ，收到上游梯度 $G = \partial L / \partial Y$ ，则： $dX = GW$ ； $dW = G^T X$ （按 batch/token 维度聚合）。这解释了为什么 backward 同时需要“上游梯度”和某些 forward activations：没有 X ，就无法准确计算 W 的梯度。

对 Transformer 来说，Attention、MLP、Norm、Embedding 都只是更复杂的局部算子；Autograd 负责把这些局部 VJP 按计算图连接起来。PyTorch 的 autograd engine 还会在一个节点收到来自多条路径的梯度时先求和，再继续向前驱节点传播。[2][3]

2. Transformer Block 内的梯度如何流动

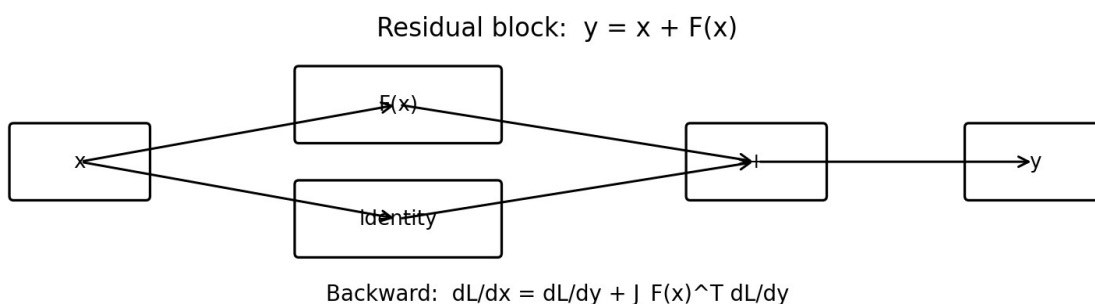


图2 Residual connection 的关键价值: gradient 同时经过 identity path 与 nonlinear branch。

残差结构 $y=x+F(x)$ 的梯度是: $dL/dx = dL/dy + J_F(x)^T \cdot dL/dy$ 。第一项是 identity gradient path, 因此即使 F 的 Jacobian 很小, 梯度仍有一条直接路径。这是深层 Transformer 能稳定训练的重要结构基础。

子模块	Backward 主要需要什么	典型风险
LM Head + CE	hidden、target/softmax statistics	大 vocab 下 logits/CE 内存与通信
RMSNorm / LayerNorm	输入统计、上游梯度	尺度异常、低精度 reduction
Attention	Q/K/V 或可重算状态、softmax 相关量	L^2 中间量、长上下文激活
SwiGLU FFN	input、gate/up activations	激活大、乘法分支放大 outlier
Residual add	上游梯度	多路径梯度相加
Embedding	token ids + output grad	稀疏/长尾 token 更新不均

Know-how 调试“某层梯度异常”时, 不要只看 parameter.grad。建议同时记录 residual stream RMS、各子层 output RMS、dgrad (对输入的梯度) 与 wgrad (对权重的梯度), 这样才能区分问题来自 forward activation、local Jacobian 还是上游 loss signal。

3. 为什么 Backward 是显存大户: Saved Activations

训练和推理最大的结构差异之一, 是训练必须为 backward 保留足够的中间状态。参数/optimizer state 的大小主要随模型参数 P 增长, 而 activation memory 还会随 microbatch、sequence length、hidden size、层数以及具体 kernel 线性或更快增长。

如果完全保存每个层的所有中间张量, 长上下文训练往往首先被 activation memory 卡住。因此现代训练框架会在“保存”和“重算”之间做系统性权衡。

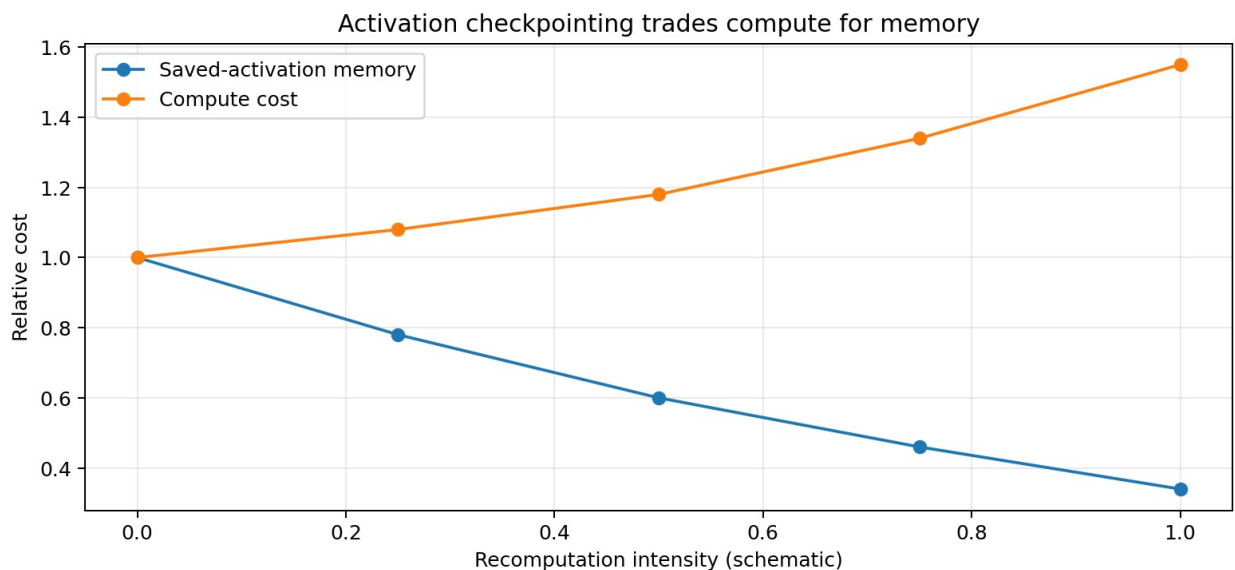


图3 Activation checkpointing / re-computation 的本质是用额外 forward compute 换更低 activation memory; 曲线为机制示意。

策略	保存什么	Backward 做什么	优点	代价
----	------	--------------	----	----

Full save	几乎所有 backward 需要的激活	直接消费已保存张量	最快	显存高
Full checkpoint	仅 checkpoint 边界	重跑大段 forward	显存低	重复计算高
Selective recompute	只重算昂贵/可重算模块的一部分	局部重算	通常更优平衡	实现更复杂
Activation offload	激活转 CPU/Host	backward 前搬回	省 GPU 显存	PCIe/NVLink 带宽压力

Chen 等 2016 年证明可用 activation checkpointing 把深网训练的 activation memory 降到次线性量级；Korthikanti 等 2022 年在大 Transformer 上进一步表明，sequence parallelism + selective activation recomputation 可以显著减少 full recomputation 的额外计算。[4][10]

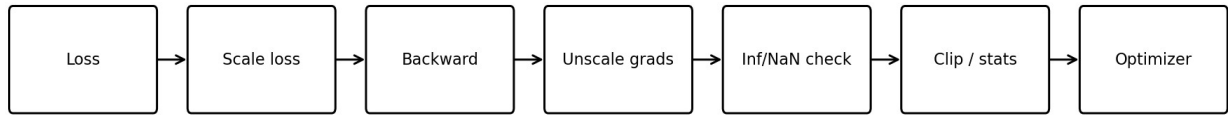
4. Gradient Accumulation：改变的是统计与同步时机

如果显存只能放 microbatch m ，但希望 global batch 为 B ，可连续执行 K 次 forward/backward，将梯度累积到同一 grad buffer，再执行一次 optimizer step。若 loss 采用 mean reduction，通常需要把每个 microbatch loss 按 K 或有效 token 数正确归一化，以保证与目标 global-batch objective 等价。

问题	错误做法	正确思路
Loss normalization	每个 microbatch 随意 mean 后直接相加	按全局有效 token/样本语义统一归一化
DDP communication	每次 backward 都 all-reduce	前 $K-1$ 个 microbatch 用 no_sync / 延迟 reduce
AMP scale	累积过程中反复改变 scale	完整 effective batch 内保持 scale 一致
zero_grad	每个 microbatch 都清零	只在 optimizer step 周期边界清零

PyTorch AMP 文档明确指出：梯度累积期间，scaled gradients 应保持同一 scale，直到完整 effective batch 结束后再 unscale/step/update；DDP 性能指南也建议 accumulation 的前 $N-1$ 次 backward 跳过不必要的 all-reduce。[5][11]

5. Mixed Precision Backward : FP16、BF16 与 Gradient Scaling



FP16 often needs loss scaling; BF16 usually relies on its wider exponent range.

图 4 典型 mixed-precision backward pipeline。

FP16 的指数范围较小，小梯度可能 underflow 为 0，大梯度也可能 overflow。经典 Mixed Precision Training 因此提出 loss scaling：先把 loss 乘 S，backward 得到 $S \cdot g$ ，再在 optimizer 前除以 S。动态 GradScaler 还会根据 inf/NaN 自动调节 S。[5][6]

模式	Backward 特征	工程建议
FP32	数值范围/精度充足	基线最稳，但显存/吞吐成本高
FP16 + scaling	需防 underflow/overflow	unscale 后再 clip / inspect
BF16	指数范围接近 FP32	通常不需要传统 loss scaling，但仍需监控异常
FP8 路线	算子/梯度/通信精度更细粒度	依赖框架 recipe 与高精度 accumulation

顺序很重要 若使用 loss scaling，需要先 unscale gradients，再做 gradient norm、clipping 或自定义梯度处理；否则看到的是被放大后的假 norm。PyTorch 官方 AMP 示例明确采用 unscale → clip → step 的顺序。[5]

6. Vanishing / Exploding Gradient 与 Gradient Clipping

深网络的梯度是许多局部 Jacobian 的连乘与残差叠加；若这些变换长期把向量缩小或放大，就会出现 vanishing/exploding gradient。Transformer 的 residual、normalization、初始化和 Pre-LN topology 都在减轻这一问题，但训练中仍会遇到 grad norm spike。

Global norm clipping 常写为：若 $\|g\|_2 > c$ ，则 $g \leftarrow g \cdot c / \|g\|_2$ 。Pascanu 等的经典分析为 norm clipping 提供了早期理论与实验依据；Megatron Core 当前也以 FP32 计算全局梯度范数，并支持跨 model-parallel process group 归约。[7][12]

指标	它告诉你什么	异常模式
Global grad norm	一次更新总体梯度尺度	突然尖峰、长期漂移
Per-layer grad norm	问题集中在哪些深度	头尾层失衡
dgrad / wgrad norm	输入梯度 vs 参数梯度	局部算子或激活异常
Zero-gradient fraction	低精度 underflow / dead path	大量突然变 0
Inf / NaN count	数值失稳	scale/LR/激活 outlier
Update-to-weight ratio	梯度最终是否导致过强更新	需与 optimizer 一起看

重要边界 Gradient clipping 是“限制一次异常梯度的破坏”，不是修复训练 recipe 的万能药。如果 clipping 长期频繁触发，应继续查 LR、初始化、数据异常、loss scale、Norm/Residual dynamics，而不是只把阈值调低。

7. Distributed Backward：梯度是计算图，也是通信图

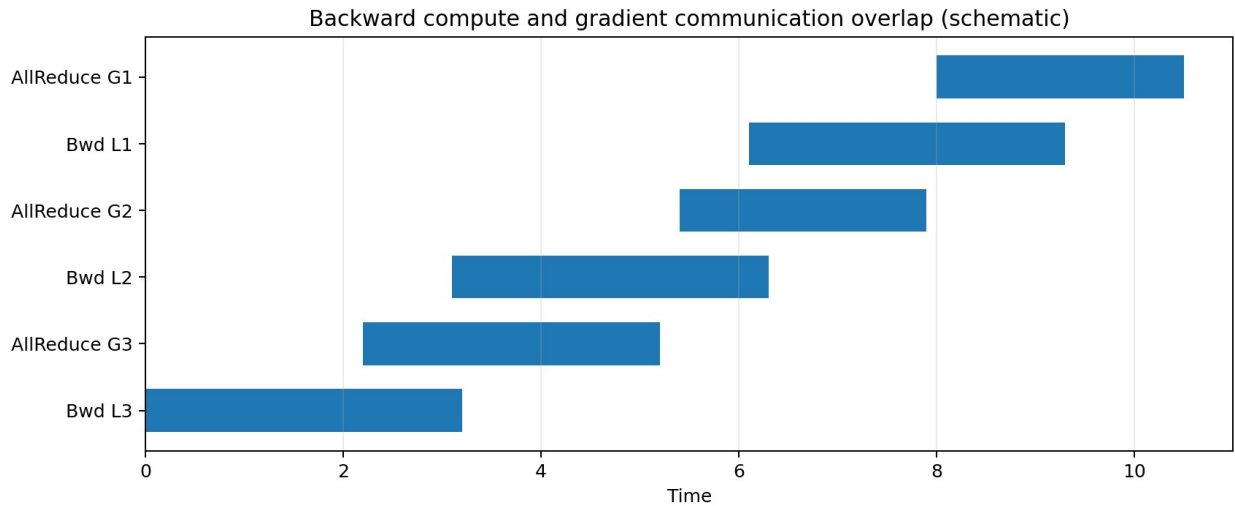


图5 DDP 的关键优化：某个 gradient bucket ready 后立即异步 reduce，使通信与更早层的 backward compute 重叠。

7.1 Data Parallel / DDP

每个 DP rank 持有模型副本并计算本地梯度。DDP 通过 autograd hooks 在梯度 bucket ready 时启动异步 all-reduce，最终让各 replica 得到一致的平均梯度。官方文档强调，这一 overlap 依赖 gradient ready order 与 bucket 设计。[11]

7.2 Tensor / Sequence Parallel

模型被切分后，局部 backward 的数学操作会天然对应通信：Megatron Core 里某些 tensor-parallel mapping 明确具有“forward copy, backward all-reduce”“forward reduce-scatter, backward all-gather”等互补语义。Linear backward 还可把 dgrad all-reduce 与 wgrad GEMM 异步重叠。[13]

7.3 Pipeline Parallel

Pipeline stage 的 backward 接收下一 stage 传回的 output_tensor_grad，并计算当前 stage 参数梯度与对前一 stage 的 input gradient。1F1B / interleaved schedule 的目标，就是让某个 microbatch 的 backward 与其他 microbatch 的 forward/communication 尽可能重叠，减少 pipeline bubble。[9][14]

7.4 ZeRO / FSDP / Distributed Optimizer

ZeRO-2/FSDP 会把 gradients 与 optimizer states 分片；ZeRO-3 进一步分片参数。Megatron distributed optimizer 的典型数据流是：backward 生成梯度 → reduce-scatter → 每个 DP rank 只保留自己负责的 fully reduced gradient shard → optimizer step → parameter all-gather。[8][15]

8. Backward Runtime 的三个“重叠”目标

重叠对象	目标	典型机制
Compute ↔ DP gradient reduce	隐藏网络延迟	DDP buckets、overlap_grad_reduce
dgrad ↔ wgrad communication	隐藏 TP 通信	async allreduce dgrad + local wgrad GEMM
Forward ↔ Backward microbatches	降低 PP bubble	1F1B / interleaved pipeline
Recompute ↔ 参数 unshard/reshard	避免重复状态搬运	FSDP checkpoint-aware hooks

这说明大规模 LLM 的 backward 不能只用“FLOPs”评估。即使数学梯度完全相同，bucket size、通信顺序、parameter sharding、recompute 粒度不同，也会导致完全不同的 wall-clock throughput 和 memory peak。

9. Gradient Noise：为什么 Batch Size 会改变训练效率

单个 mini-batch gradient 只是总体期望梯度的随机估计。batch 越大，方差通常越低，但超过某个规模后继续增大 batch 的样本效率收益会递减。McCandlish 等提出 gradient noise scale，用于估计“最大有用 batch size”及 compute-efficiency / time-efficiency trade-off。[16]

视角	小 Batch	大 Batch
Gradient variance	高	低
Step 数	多	少
Data efficiency	通常较好	超过临界规模后递减
并行效率	低一些	更容易吃满大量设备
调参耦合	LR/accumulation 更敏感	LR schedule / warmup 同样需匹配

Know-why Gradient accumulation 可以模拟更大的 effective batch，但它不会凭空提供更多并行算力；它主要解决显存问题。真正的数据并行扩大 batch 则会改变每一步同时使用的数据量和 wall-clock 并行度。

10. Backward Observability：生产训练该看什么

层级	建议指标	目的
Global	loss、global grad norm、clip fraction、Inf/NaN	判断整体稳定性
Layer	per-layer grad norm、activation RMS、dgrad/wgrad ratio	定位异常深度
Tensor type	QKV/MLP/Norm/Embedding grad stats	识别模块级热点
Precision	zero fraction、overflow events、loss-scale history	诊断低精度问题
Distributed	grad bucket ready time、all-reduce exposed time	诊断通信 overlap
Memory	saved activation bytes、recompute FLOPs、peak HBM	选择 checkpoint policy
Batch statistics	effective tokens/step、noise-scale proxy	连接梯度质量与 batch size

10.1 推荐的故障定位顺序

- 先确认 loss 与 labels/mask/reduction 正确；
- 再看 global grad norm 与 Inf/NaN，确定是否是数值失稳；
- 按层查看 activation RMS 与 grad norm，定位最先异常的位置；
- 对该层拆 dgrad / wgrad，判断异常来自上游还是局部算子；
- 检查 AMP scale、gradient clipping、microbatch normalization；
- 最后检查 distributed reduction 是否遗漏、重复或缩放错误。

11. 严格 Backprop Ablation：怎样避免假结论

实验问题	必须固定	主要观测
Checkpointing 策略	模型、batch、seq length、kernel	peak memory、tokens/s、MFU、loss trajectory
Grad clipping 阈值	LR、optimizer、data order	clip fraction、spike recovery、final loss
Accumulation steps	global tokens/step、loss normalization	梯度等价性、throughput
FP16/BF16	模型、optimizer、batch	overflow/underflow、convergence、MFU
DDP bucket/overlap	并行拓扑、global batch	exposed communication、step time
Recompute granularity	memory budget、seq length	额外 FLOPs、HBM、wall time

实验原则 Backprop 优化通常存在“数学等价但系统不等价”的情况。首先验证梯度/损失等价性，再比较吞吐和显存；不要只看 tokens/s，也不要只看 peak memory。

12. Production Backward Blueprint

阶段	推荐做法
1. Correctness baseline	FP32/BF16 小规模 run；核对 loss/reduction/gradient 数值
2. Mixed precision	优先 BF16；FP16 明确配置 loss scaling 与 overflow policy
3. Accumulation	以 global valid tokens 定义等价目标；减少无效 DP sync
4. Activation memory	先 selective recompute，再 full recompute；长上下文评估 offload
5. Distributed overlap	DP reduce 与 backward 重叠；TP dgrad comm 与 wgrad GEMM 重叠
6. Gradient safety	global norm、clip fraction、Inf/NaN、zero fraction 常驻监控
7. Scale-out	确认 PP/TP/FSDP 特殊梯度在 optimizer 前全部 finalize/sync
8. Regression	每次 kernel/parallelism 调整都做 gradient parity + convergence proxy

13. Know-Why / Know-How 总结

问题	Know-Why	Know-How
为什么 reverse-mode 适合 LLM?	scalar loss、海量参数，反向一次可得全部参数梯度	使用 VJP/autograd，不显式构造 Jacobian
为什么需要保存 activation?	wgrad/dgrad 依赖 forward 中间状态	checkpoint / selective recompute / offload
为什么 residual 对深层训练重要?	identity path 避免梯度只能穿过连续 nonlinear Jacobian	监控 residual RMS 与 branch grad
为什么 gradient accumulation 容易出错?	它改变了 loss normalization 与同步时机	按 global valid tokens 归一化；只在周末 sync/step
为什么 FP16 要 loss scaling?	小梯度可能 underflow	scale → backward → unscale → clip/step
为什么 clipping 不能解决根因?	只限制最终 norm，不修复产生异常的 dynamics	监控 clip frequency，并追查 LR/数据/激活
为什么 distributed backward 很复杂?	梯度产生顺序决定通信何时能启动	bucketize、overlap、reduce-scatter、1F1B
为什么 Backprop 和 Optimizer 要分开理解?	Backprop 求“方向/责任”，Optimizer 决定“如何沿此信息更新参数”	先验证 gradient correctness，再调 optimizer

参考资料与证据等级

1. Rumelhart, Hinton, Williams. Learning representations by back-propagating errors. Nature, 1986. [经典基础]	2. PyTorch Documentation. torch.autograd / Automatic Differentiation. Current documentation. [官方实现]
3. PyTorch. Overview of PyTorch Autograd Engine. [官方机制说明]	4. Chen et al. Training Deep Nets with Sublinear Memory Cost. arXiv:1604.06174, 2016. [经典论文]
5. PyTorch Documentation. Automatic Mixed Precision examples / gradient accumulation. [官方实现]	6. Micikevicius et al. Mixed Precision Training. arXiv:1710.03740, 2017. [经典论文]
7. Pascanu, Mikolov, Bengio. On the difficulty of training Recurrent Neural Networks. arXiv:1211.5063. [经典梯度分析]	8. Rajbhandari et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. arXiv:1910.02054. [系统论文]
9. Narayanan et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. arXiv:2104.04473. [系统论文]	10. Korthikanti et al. Reducing Activation Recomputation in Large Transformer Models. arXiv:2205.05198. [系统论文]
11. PyTorch Documentation. DistributedDataParallel / DDP design note. [官方实现]	12. NVIDIA Megatron Core. optimizer.clip_grads. [官方实现]
13. NVIDIA Megatron Core. tensor_parallel.mappings / tensor_parallel.layers. [官方实现]	14. NVIDIA Megatron Core. pipeline_parallel.schedules / combined 1F1B. [官方实现]
15. NVIDIA Megatron Core. Distributed Optimizer / FSDP / finalize_model_grads. [官方实现]	16. McCandlish et al. An Empirical Model of Large-Batch Training. arXiv:1812.06162, 2018. [经典实验]
17. PyTorch Foundation. Current and New Activation Checkpointing Techniques in PyTorch, 2025/2026 update. [官方工程资料]	18. Megatron Bridge. Activation Recomputation guidance. Current documentation. [官方工程资料]

本册结论 Backpropagation 的数学核心几十年没有改变，但 LLM 时代的主要创新集中在“如何执行同一个梯度计算”：少存激活、选择性重算、低精度数值保护、microbatch 累积、梯度分片，以及把通信隐藏在 backward compute 后面。下一册进入 Optimizer：梯度已经得到后，AdamW 等算法如何把它转成真正的参数更新。